

# Autonomous Search and Rescue

Project B - IA712 Mobile Robotics, Final Project

## Team RobotZ

Julien GIMENEZ   Hugo FANCHINI   Paul CINTRA   Yimou ZHANG

<https://github.com/YiZeems/autonomous-search-and-rescue>

IA712 - Session L18

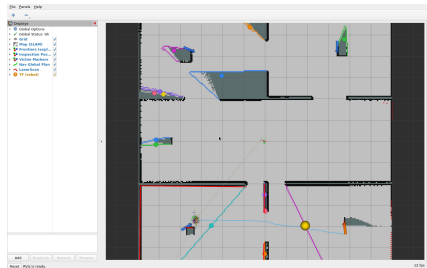
ROS 2 Humble · Ignition Gazebo Fortress · TurtleBot 4 · Nav2 · slam\_toolbox · BehaviorTree.CPP

# The mission: autonomous search & rescue

**Scenario:** a robot explores an unknown disaster zone, finds “victims” (AprilTags), and reports their global coordinates - *no human input*.

**Requirements we satisfy:**

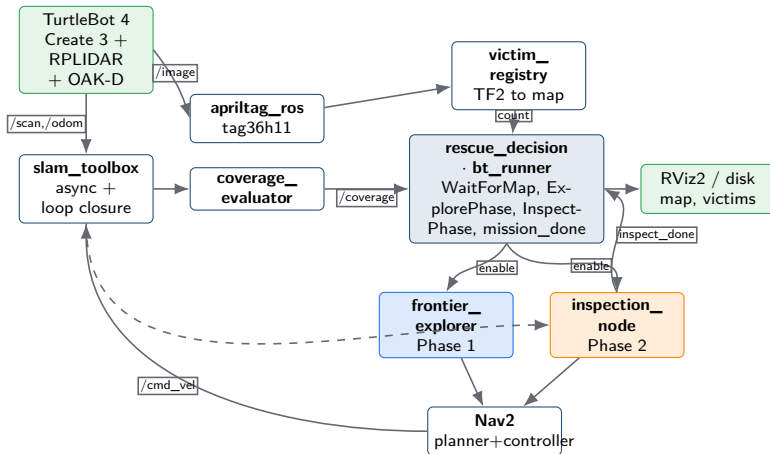
- Autonomous frontier exploration, map > 90 %
- Camera detection to map via tf2
- Map quality / loop closure (SLAM)
- **Decision = Behavior Tree, not FSM**
- One-click launch
- Bonus: greedy vs. information-gain



Live RViz: map + 4 victims + algorithms.

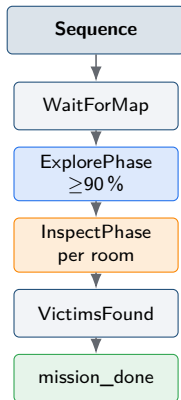
**Result:** one continuous autonomous run - **97.35 % coverage, 4/4 victims.**

## System architecture - four ROS 2 packages

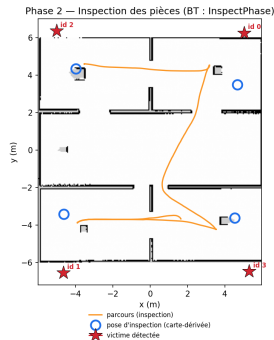
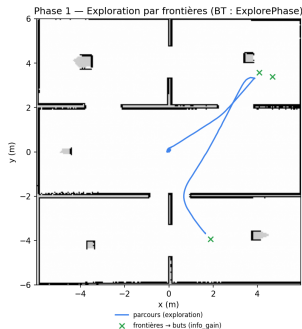


The **Behavior Tree orchestrates**: it gates the Phase 1 explorer and the Phase 2 inspector via `/mission/*` topics; the Python nodes do the work. SLAM feeds the map to exploration, coverage, Nav2 and inspection.

# Two-phase mission orchestrated by the Behavior Tree



Algorithmes en action — mission orchestrée par le Behavior Tree

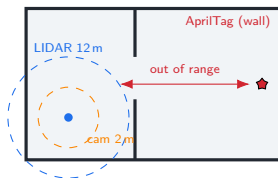


**Phase 1** (blue) frontier exploration builds the map; **Phase 2** (orange) inspects each room at *map-derived* poses (no victim coordinates) to catch every wall tag.

Why two phases? Pure exploration maps rooms from doorways but the camera never gets within 2 m of the wall tags, so it caps at  $\sim 2$  victims.

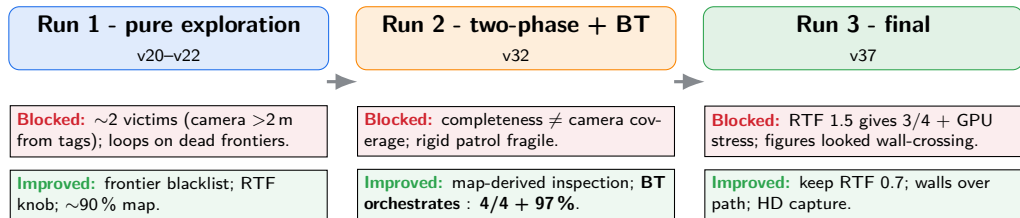
# Theoretical foundations

- **SLAM + loop closure** - consistent online map.
- **Frontier exploration** - free/unknown boundary; *information-gain*  
 $g(f) = \text{gain} - \lambda \text{cost}$ . Dead-frontier blacklist (Nav2 failure, or  $3\times$  with no gain, blacklisted).
- **Path planning** - Dijkstra/NavFn vs. A\*; NavFn kept (tolerant in tight space).
- **TF2** - project camera detection to global map (project requirement).
- **Behavior Tree** - reactive, composable; memory Sequence, not an FSM.



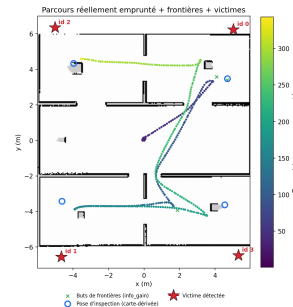
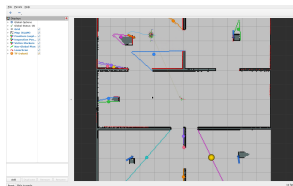
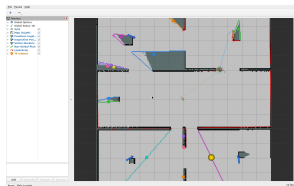
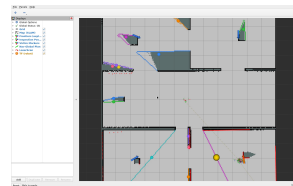
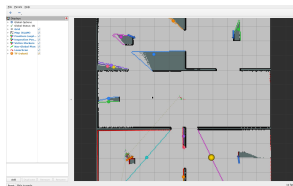
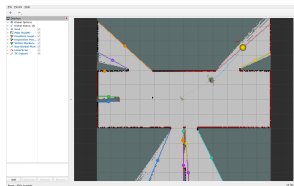
The camera-vs-LIDAR gap that the inspection phase closes.

## The decision journey - three runs



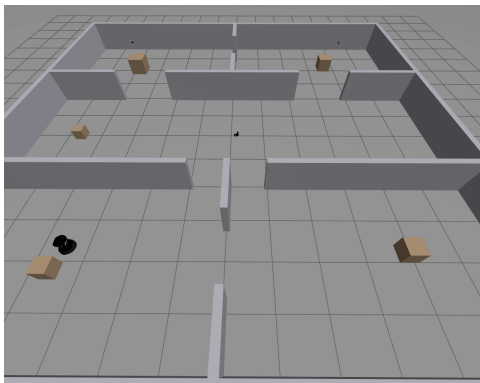
Two hidden root causes: Nav2 cost discouraged *entering* rooms (coverage); a collapsed RTF *starved the camera* (detection). The 2-phase BT then made 4/4 reliable.

# Results - live algorithms in RViz

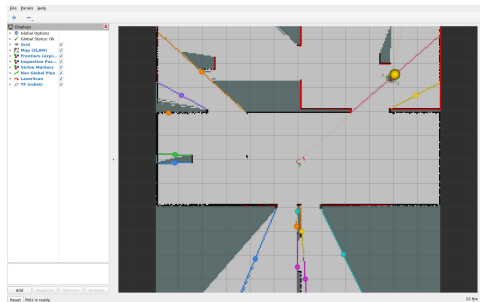


Exploration, inspection, four victims; bottom-right: the real map-frame path (walls drawn over it, so it visibly threads the doorways - no segment crosses a wall).

## Seeing the algorithm: Gazebo vs RViz



**Gazebo** - the simulated arena: walls, victims (AprilTags) and the robot.

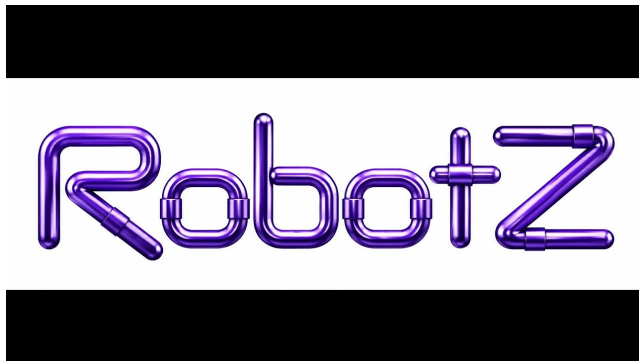


**RViz** - the SLAM map + *frontier cells coloured per cluster*, the goal frontier (yellow) and the Nav2 plan (blue).

Each frontier *cluster* has its own colour, the robot targets the best utility  $g = \text{gain} - \lambda \text{cost}$ , and you see the coloured rim eat into the unknown up to  $> 90\%$ . Captures: gazebo\_capture.mp4 & rviz\_capture.mp4.

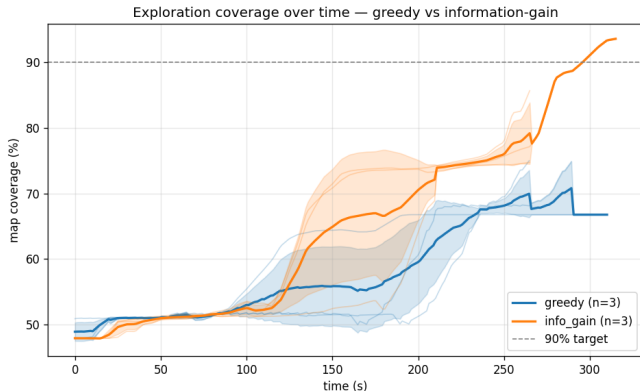
## Demonstration

### Démo RobotZ



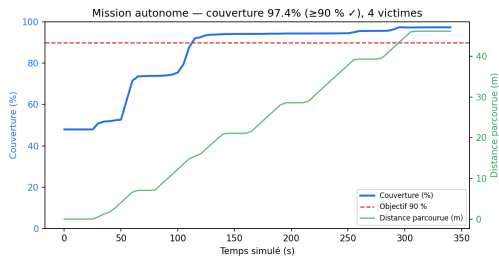
Full-mission video (autonomous run, one click): <https://www.youtube.com/watch?v=BKXprWFloFs>

## Bonus: map coverage - greedy vs information-gain



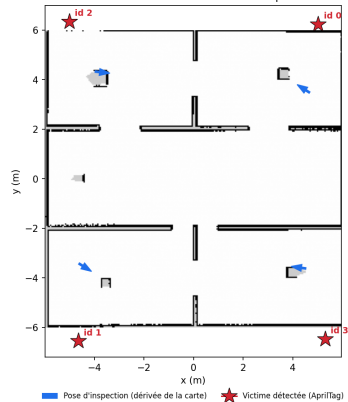
Benchmark  $n=3$ : **information-gain** (orange) reaches the 90 % target; **greedy** (blue) plateaus at ~67–70 % and never reaches it - it commits to opening whole rooms (more unknown per metre), whereas greedy nibbles locally and stalls.

# Results - coverage, map and conformity



Coverage & path vs. time, 90 % target.

Carte SLAM finale — 4 victimes + tournée d'inspection autonome



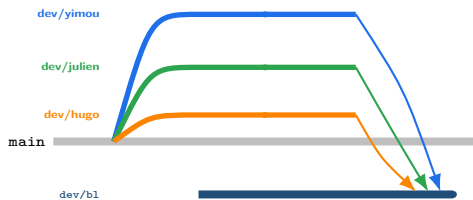
Final map: 4 victims + autonomous inspection tour.

**97.35 % coverage · 4/4 victims · annotated map · logged trajectory - one click, fully autonomous.**

# Team and collaboration

Per-member dev/\* branches merged into dev/b1 (integration):

- **Yimou** - exploration, SLAM/Nav2, perception core
- **Julien** - BT decision, infra (WSL/GPU/RTF), results, integration
- **Hugo** - world / arena / AprilTag targets
- **Paul** - perception/detection support, testing



Three workstreams to b1 integration.

## Lessons learned

- **Measure before prescribing** - “obvious” causes (memory, Windows Update) were wrong twice; event logs found the real ones (GPU TDR, a starved camera).
- **One fix reveals the next** - victim detection crossed six stacked causes.
- **Robust beats rigid** - adaptive exploration + 360° sweep beat a fixed patrol; the inspector also recovers a pose another machine cannot reach (wait for the costmap, fall back into the room) - *cross-machine robustness*.
- **Conformity by design** - a BT that *orchestrates* (not just supervises) meets the assignment in spirit, not only in letter.

Timeline: L13 kick-off · L14 architecture · L15 nodes/SLAM · L16 BT+perception · L17 integration/benchmark · L18 two-phase, final run, report.

## Thank you - Questions?

We warmly thank **Zhi Yan**, Teacher-Researcher at ENSTA,  
for the IA712 course and guidance.

<https://github.com/YiZeems/autonomous-search-and-rescue>

Julien GIMENEZ · Hugo FANCHINI · Paul CINTRA · Yimou ZHANG